第 17 章練習解答　分散式系統

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS17-E01　Speedup 與故障域

1. T6=30+90/6+6=51s；S6=120/51≈2.353；忽略協調的界120/30=4。node增加不刪串行與協調。
2. 獨立時.02^3=.000008=8×10^{-6}；共rack可有相關失效，不能再相乘。metadata不可用也可能讓有data副本卻無法服務，因此此數字不是整體SLO。

檢查。 可靠度分母與共同故障前提明列。

OS17-E02　搬資料還是搬計算

1. 搬資料512/64+.010=8.010s，尚需local compute；remote .200+2/64+.010=.24125s。比較含各模型指定成本，不把傳輸時間當總application時間。
2. 不是，完整process狀態還有PC/register、memory、open resources與相容ABI；remote啟動另一份程式可屬computation placement。

檢查。 MiB與MiB/s一致。

OS17-E03　BDP 與 window

1. BDP=200×10^6×.05=10000000bits=1250000B≈1220.70KiB。window bound=131072×8/.05=20.97152Mbit/s；低於link200M。
2. 不會，window/RTT仍20.97152Mbit/s。須解除在途資料限制且其他瓶頸也允許，才可能利用較高linkrate。

檢查。 bytes×8才變bits，RTT不是單向delay。

OS17-E04　DNS cache 與命名

1. t150舊cache仍可能返回.20、剩TTL150；t300到期後需重新query成功才取得當時值，不能假定網路失敗時已有新值。DNS依zone delegation，不要求每label一server。
2. 都不能直接等同。domain name映地址/記錄，IP為路由端點位址，port參與transport demux；user/authentication與service object identity另有機制。

檢查。 修改權威不立刻撤所有舊cache。

第 17 章練習解答　分散式系統 （續）

OS17-E05　Routing 與 packet pipeline

1. A初始化B∞；先確定D1，B6；再確定C2，B4；確定B，A-C-B cost4。刪C-B後A-D-B cost6。動態路由需偵測/收斂，不是瞬時必不丟。
2. 每packet每hop8ms，pipeline共(4+2−1)8=40ms；整message每hop32ms，共64ms。若有headers需重新算每packet長度，不能忽略分割成本。

檢查。 Dijkstra前提非負固定cost；pipeline前提固定link/forwarding模型。

OS17-E06　IPv4／ARP／Ethernet 短封包

1. A ARP next-hop192.0.2.1，得R_L；第一frame A→R_L，第二R_R→B；兩hop IP endpoints均192.0.2.10→198.51.100.20。R需在右LAN解析B，並減TTL、更新IP checksum。
2. IP44B，Ethernet data最少46故pad2；MAC frame14+46+4=64B。加preamble/SFD8與96bit IFG=12 byte-times，共84byte-times；100Mbit/s占84×8/10^8=6.72μs。IFG非IP資料。

檢查。 無tag/無options/100Mbit模型；6B MAC不是任意2B地址。

OS17-E07　Failure detector 的不可分辨性

1. 一條B在ping前crash；另一條B仍活、request或reply延到201ms之後。到200ms A都只看到send和timer，無reply，故無法只由此history正確區分。
2. 在前次可即時到達、後次取最慢時，最大.5+.1+.05=.65s。超過此界可懷疑模型/鏈路/節點有問題，但若原delay bound失效，原無誤判保證也失效。

檢查。 unbounded async的timeout不是crash或任何物理故障的證明。

OS17-E08　Fencing 與 replay

1. partition下A仍可寫，雙主。須由正確協調服務核發epoch，storage原子安裝43並逐write驗token；所有42寫拒，新owner才可運行。兩邊自己各加一不滿足唯一核發前提。
2. 把counter++與durable id17→result11同交易提交。retry查table回11。若更新counter後尚未記id便crash，重試可再加；先記id後未改counter則遺失效果。兩狀態必原子/可恢復一致。

檢查。 at-most-once不保證client一定知道結果或外部副作用受同交易覆蓋。

第 17 章練習解答　分散式系統 （續）

OS17-E09　DFS naming 與 cache 成本

1. 改vol9→location，不改穩定path/FID；刪file2再建新檔需換generation，否則舊FID可能錯指新檔。location cache要按版本/失效更新，不能永久當有效。
2. 20×6+80×.2=136ms，traffic20×4=80KiB。每read額外1ms再加100ms，合236ms；不能因data命中就假定完全無network成本。

檢查。 穩定identity與位置分開；cache validation有自己的成本/語義。

OS17-E10　Consistency 的時間與信任

1. 可返回v0，因尚未再驗；故該模型不是每read都見已完成write的強語義。不能把client-initiated periodic validation與server強同步失效混同。
2. server需在完成write前收回衝突讀許可或等受控lease失效，client也不得在許可失效後繼續無限用cache。只有發一封失效訊息但不確認、不限lease，partition時不能證舊client停止讀。

檢查。 應用已讀進自有buffer的bytes不會因callback自動變更。

OS17-E11　NFSv3 unstable 與 verifier

1. 不能把先前UNSTABLE當已保留；需重送尚未取得stable保證的D，再取得對應穩定結果並核verifier。正常同instance COMMIT成功且verifierX一致時，才依契約清該批重送狀態。
2. 都不是。核心READ/WRITE攜filehandle/offset等不需同open session狀態，server仍能cache/duplicate-request cache；WRITE有不同stable modes。durability、visibility與去重需分開判。

檢查。 固定NFSv3/RFC1813，不套NFSv2或v4的不同狀態模型。

OS17-E12　HDFS 歷史版與 failover capacity

1. 64、64、22MiB共3blocks，副本payload450MiB，不含metadata/checksum/配置額外。NameNode管namespace/location，client資料在DataNodes傳輸，不把所有data經NameNode。Secondary NameNode負checkpoint，不是自動hot standby。
2. 容量100<150，至少50requests/s不能穩態消化。能接手不等於保原吞吐/SLO，須預留餘裕/降級/限流。副本數也不保證metadata/network不失效。

檢查。 副本payload、物理配置與metadata可靠度分開。

第 17 章練習解答　分散式系統 （續）

原創驗算程式附錄: distributed_models.py

```
from fractions import Fraction as F
from heapq import heappush,heappop
def distance(graph,start):
 q=[(0,start)];d={start:0}
 while q:
  cost,u=heappop(q)
  if cost!=d[u]:continue
  for v,w in graph[u].items():
   c=cost+w
   if v not in d or c<d[v]:d[v]=c;heappush(q,(c,v))
 return d
def increment(state,requests,rid):
 # One atomic transaction in this pure model. Real storage must implement it.
 if rid in requests:return state,requests,requests[rid]
 state+=1;requests={**requests,rid:state};return state,requests,state

class NFSModel:
 def __init__(self):self.stable={};self.ram={};self.verifier=1
 def write_unstable(self,offset,data):
  self.ram[offset]=data
  return len(data),self.verifier
 def commit(self):
  self.stable.update(self.ram);self.ram.clear();return self.verifier
 def crash(self):
  self.ram.clear();self.verifier+=1

class FencedStore:
 def __init__(self):self.epoch=42;self.value=None
 def install(self,new_epoch):
  assert new_epoch>self.epoch
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 17 章練習解答　分散式系統 （續）

```
  self.epoch=new_epoch
 def write(self,epoch,value):
  if epoch!=self.epoch:return False
  self.value=value;return True
if __name__=='__main__':
 g={'A':{'C':2,'D':1},'C':{'A':2,'B':2},'D':{'A':1,'B':5},'B':{'C':2,'D':5}}
 assert distance(g,'A')['B']==4;del g['C']['B'];del g['B']['C'];assert distance(g,'A')
['B']==6
 assert (4+2-1)*1040*8/1_000_000==.0416
 assert 14+20+20+1000+4==1058 and 1058+8+12==1078
 assert 14+max(46,20+20+4)+4==64
 assert 84*8/100_000_000==.00000672
 assert F(65536*8,1)/F(40,1000)==F(13107200,1)
 state,seen,result=increment(10,{},17);assert(state,result)==(11,11)
 state,seen,result=increment(state,seen,17);assert(state,result)==(11,11)
 assert F(20*6)+80*F(2,10)==136
 s=NFSModel();pending=b'D'*4096
 count,old=s.write_unstable(0,pending);assert count==4096 and s.stable=={}
 s.crash();new=s.commit();assert new!=old and s.stable=={}
 # Retain pending when generation mismatches; resend and commit new instance.
 count,new=s.write_unstable(0,pending);assert s.commit()==new
 assert s.stable[0]==pending
 s.crash();assert s.stable[0]==pending
 f=FencedStore();assert f.write(42,'old-owner-before-fence')
 f.install(43);assert not f.write(42,'stale') and f.write(43,'new-owner')
 assert f.value=='new-owner'
 print('route costs 4/6; pipeline41.6ms; frame1058/wire1078; shortframe64; BDP-
window13.1072Mbit/s')
 print('atomic retry id17: counter11 after duplicate; cache136ms; NFS crash loses RAM,
verifier mismatch/resend restores D; old epoch rejected')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 17 章練習解答　分散式系統 （續）

原創驗算程式附錄: rpc_retry.py

```
"""Model retry after a lost response in one serial, non-crashing server.

The in-memory dictionary is not durable and there is no concurrent access.
This example does not prove exactly-once effects across crashes.

Run:
    python rpc_retry.py
"""


class Server:
    def __init__(self) -> None:
        self.commits: dict[str, str] = {}
        self.executions = 0

    def create_once(self, request_id: str, name: str) -> str:
        if request_id in self.commits:
            return self.commits[request_id]
        self.executions += 1
        result = f"object:{name}"
        self.commits[request_id] = result
        return result


def main() -> None:
    server = Server()
    first = server.create_once("req-17", "notes")
    retry = server.create_once("req-17", "notes")
    assert first == retry
    assert server.executions == 1
    print("result", retry, "executions", server.executions)


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study